

Y-Dude – Server- und Backend-Lasttest

SPEAK LOCAL. CONNECT GLOBAL. · Laststufen 250 / 500 / 750 · Stand 15.08.2026

TEIL 1 – GEMESSENE ERGEBNISSE

1. Executive Summary (gemessen)

Stufe	Requests	Req/s	Fehler	Timeouts	Q	p50	p90	p95	p99	max	Status
250	14 979	329,8	0 (0,00 %)	0	755 ms	838 ms	1 015 ms	1 423 ms	3 642 ms	14 621 ms	stabil
500	15 256	329,3	0 (0,00 %)	0	1 498 ms	1 347 ms	2 104 ms	2 301 ms	2 892 ms	12 498 ms	stabil
750	14 854	316,2	0 (0,00 %)	0	2 328 ms	2 202 ms	2 981 ms	3 115 ms	3 797 ms	14 772 ms	Grenzbereich

45 089 Requests gesamt, ausschließlich HTTP 200. Keine Verbindungs- und keine Datenbankfehler.

2. Testbedingungen und Methodik

Punkt	Wert
Umgebung	laufende Anwendung (TanStack Start / SSR, Projektumgebung), produktive Cloud-Datenbank
Dauer je Stufe	45 s Dauerlast + 15 s Abkühlphase (kein kurzer Peak)
Request-Timeout	20 s · Verbindungen mit Keep-Alive
Routen-Mix	/ (30 %), /post/<id> ×3 (35 %), /auth (10 %), Rechtsseiten (20 %), /api/public/cache-metrics (5 %)
Schreiboperationen	keine – ausschließlich lesende GET-Anfragen
Nicht getestet	angemeldete Pfade (Feed, Profil, Arena, Globe, Messenger, Suche) sowie Likes, Kommentare, Follows, Beiträge, Push, KI-Moderation – sie würden echte Nutzerdaten verändern

3. Backend, Cache und Ressourcen je Stufe (gemessen)

Stufe	gleichz. verarbeitet (Spitze)	Q Serverzeit	max Serverzeit	Event-Loop-Lag	RAM (RSS)	CPU im Lauf	Cache-Treffer	gebündelt	DB-Abfragen
250	139	77 ms	828 ms	2 ms	1 742 → 2 412 MB	48,72 s	6 451	131	3
500	227	84 ms	2 438 ms	2 ms	2 412 → 2 482 MB	47,72 s	5 697	214	3
750	256 (Deckel)	86 ms	2 438 ms	2 ms	2 482 → 2 482 MB	47,77 s	5 175	285	3

Cache-Trefferquote 99,94–99,95 %. Kein weiterer Speicheranstieg bei 750 → kein Hinweis auf Memory Leak.

4. Datenbank während des Tests (gemessen)

Kennzahl	Wert
Datenbank / PgBouncer	up / up
Verbindungen	21 / 60 (niedrig)
Connection-Pool-Clients	4 / 200 (niedrig)
Arbeitsspeicher	43 %
Datenträger	18 %
Datenbankgröße	47,1 MB
WAL-Größe	128 MB
Neustarts / OOM-Kills	0 / 0
Datenbankfehler im Test	0

Kennzahl	Wert
Abfragezeiten (Statistik)	Ø 0,06–2,86 ms · max 17,69 ms

5. Vergleich mit früheren Tests

Ein Lasttest mit Datum **02.08.2026 existiert im Projekt nicht** (kein Dokument, kein Protokoll, keine Messdatei). Verglichen wird daher mit den dokumentierten Tests vom 05.08., 13.08. und 14.08.2026. Fehlende Werte sind als „nicht verfügbar“ ausgewiesen und wurden nicht ergänzt.

Kennzahl	02.08.2026	05.08.2026 (gemessen)	13.08.2026	15.08.2026 (dieser Test)	Veränderung
250 – Ø	nicht verfügbar	1 143 ms	nicht ausgewiesen	755 ms	–34 %
250 – p95	nicht verfügbar	1 887 ms	nicht ausgewiesen	1 423 ms	–25 %
250 – p99	nicht verfügbar	nicht verfügbar	nicht verfügbar	3 642 ms	kein Vergleich
250 – Req/s	nicht verfügbar	202	nicht verfügbar	329,8	+63 %
500 – Ø	nicht verfügbar	nur modelliert	ca. 2,9 s	1 498 ms	–48 %
750 – p90	nicht verfügbar	nicht ausgewiesen	ca. 8,4 s	2 981 ms	–65 %
Fehler / Timeouts	nicht verfügbar	0 %	0 / 0	0 / 0	unverändert

Der Bericht vom 05.08.2026 weist Stufen ab 500 selbst ausdrücklich als modelliert aus – diese Zellen sind daher kein Messvergleich.

Direkter Vergleich mit der Messung vom 14.08.2026 (gleiche Umgebung, aber nur eine Route und nur eine Abfragemenge):

Stufe	14.08. p50 / p90 / p95	15.08. p50 / p90 / p95	Requests 14.08.	Requests 15.08.
250	871 / 1 604 / 1 683 ms	838 / 1 015 / 1 423 ms	250	14 979
500	1 134 / 2 193 / 2 511 ms	1 347 / 2 104 / 2 301 ms	500	15 256
750	1 938 / 3 184 / 3 305 ms	2 202 / 2 981 / 3 115 ms	750	14 854

Trotz rund 20-fachem Anfragevolumen bleibt die Latenz auf gleichem Niveau – die Anwendung hält Dauerlast durch.

6. Flaschenhalse und Fehleranalyse (gemessen)

Befund	Bewertung
Parallelitätsdeckel 256 gleichzeitig verarbeiteter Anfragen	Hauptflaschenhals: oberhalb davon steigt nur die Wartezeit, nicht der Durchsatz
Durchsatzplateau ~316–330 Requests/s	in allen drei Stufen nahezu identisch
SSR-Rendering je Anfrage (Ø 77–86 ms Serverzeit)	CPU-Arbeit, unter Last stabil
Datenbank	kein Engpass: 3 Abfragen je Laststufe, Pool 4/200
Event-Loop-Lag 2 ms konstant	keine blockierenden Operationen
Fehler / Timeouts / Verbindungsfehler	0 / 0 / 0 – Fehlergrenze im Test nicht erreicht
Ausreißer max. 12,5–14,8 s	Warteschlangen-Ausreißer beim Stufenstart, unter dem 20-s-Timeout, keine Fehler

TEIL 2 – TECHNISCHE HOCHRECHNUNG UND EMPFEHLUNG (KEINE MESSWERTE)

7. Belastungsgrenze und Kapazität

Bereich	Einschätzung	Grundlage
stabile Last	bis ca. 300 gleichzeitige Nutzer (p95 < 1,5 s)	gemessen
erhöhte Latenz	500 Nutzer, p95 2,3 s, fehlerfrei	gemessen
kritische Last / Komfortgrenze	750 Nutzer, p95 3,1 s, fehlerfrei	gemessen
Fehlergrenze	nicht erreicht	gemessen
1 000 Nutzer	p95 ca. 4,0–4,5 s, voraussichtlich 0 % Fehler	Hochrechnung
2 500 Nutzer	p95 ca. 10–12 s, erste Timeouts möglich	Hochrechnung
5 000 Nutzer	p95 ca. 20 s+, deutliche Timeout-Quote	Hochrechnung

Bereich	Einschätzung	Grundlage
10 000 Nutzer	> 20 s, hohe Fehlerquote ohne weitere Instanzen	Hochrechnung

Hochrechnung aus dem gemessenen Durchsatzplateau (~330 Req/s) und dem Parallelitätsdeckel. Keine zugesicherte Kapazität.

8. Empfehlungen nach Priorität

Prio	Optimierung	Problem / Ursache	Erwarteter Effekt	Aufwand	Risiko	in Lovable?
A	Vier häufige Sitzungsabfragen bündeln (ad_preferences, hashtag_follows, connections, feed_learned_weights)	5 000–8 000 Einzelabfragen in der Statistik	–60 bis –80 % Abfragen beim Sitzungsstart	gering	gering	ja
A	HTTP-Cache-Header auf öffentliche Routen anwenden	jede Anfrage rendert SSR neu	SSR-Last sinkt stark, Auslieferung aus dem CDN	gering	gering	ja
A	Cache-Lebensdauer für Stammdaten anheben	slang_tags-Sortierabfrage Ø 1,97 ms / max 17 ms	weniger DB-Arbeit bei Spitzen	gering	gering	ja
B	Statische Rechtsseiten vorrendern	20 % der Anfragen rendern reinen Text	mehr freie Parallelität für dynamische Routen	mittel	gering	ja
B	Größere Cloud-Instanz	Parallelitätsdeckel 256	proportional mehr Durchsatz, niedrigere p95	gering	gering (Kosten)	ja
B	Bildoptimierung und Langzeit-Cache für Medien	signierte URLs je Anfrage, keine Varianten	schnellerer Seitenaufbau	mittel	mittel	ja
C	Geteilter Cache über alle Instanzen	Trefferquote gilt je Instanz	konstante Trefferquote beim horizontalen Skalieren	hoch	mittel	externe Infrastruktur
C	Read-Replica / Trennung Lese- und Schreibpfad	ein Datenpfad für alles	mehr Leseparallelität	hoch	mittel	externe Infrastruktur
C	Eigene Queue-Infrastruktur (Moderation, Push)	derzeit Warteschlangen in der Datenbank	Entkopplung bei Massenergebnissen	hoch	mittel	externe Infrastruktur

9. Skalierung innerhalb von Lovable

Bereich	Status / Möglichkeit
Backend vertikal	Instanzgröße erhöhen – direkter Hebel auf den Deckel von 256
Backend horizontal	plattformseitig; Cache wirkt dann je Instanz
Datenbank	Compute und Datenträger getrennt regelbar – aktuell nicht nötig (43 % RAM, 18 % Disk)
Connection Pooling	vorhanden (PgBouncer), 4/200 belegt – kein Handlungsbedarf
Caching / API-Caching / CDN	Server- und Client-Cache aktiv; Cache-Header konsequent auf öffentliche Routen anwenden
Query-Optimierung	Bündelung der vier häufigen Sitzungsabfragen (Priorität A)
Rate Limiting	möglich – schützt, erhöht aber die Kapazität nicht
Load Balancing	plattformseitig, kein eigener Proxy nötig
Asynchron / Background Jobs / Queues	vorhanden: Moderation, Push, Zähler laufen außerhalb des Request-Pfads
Statische Assets, Bildoptimierung, Lazy Loading	größter Frontend-Hebel, unabhängig vom Backend
Trennung stark/schwach belasteter Funktionen	öffentliche Leserouten cachebar machen, angemeldete Pfade dynamisch lassen
Externe Infrastruktur sinnvoll ab	dauerhaft mehr als ca. 2 500 gleichzeitige Nutzer bzw. Bedarf an geteiltem Cache / Read-Replicas

10. Fazit

Die zuletzt durchgeführten Optimierungen sind messbar wirksam. Bei 250 gleichzeitigen Nutzern sank die durchschnittliche Antwortzeit gegenüber der letzten vergleichbaren Messung (05.08.2026) von 1 143 ms auf 755 ms, p95 von 1 887 ms auf 1 423 ms; der Durchsatz stieg von rund 202 auf rund 330 Requests pro Sekunde. Bei 750 Nutzern liegt p90 statt bei ca. 8,4 s (13.08.2026) bei 2 981 ms. Der Caching Layer reduziert die Datenbankarbeit auf drei Abfragen je Laststufe,

Event-Loop-Lag und Speicherverbrauch bleiben unter Dauerlast stabil, und in 45 089 Requests trat kein Fehler, kein Timeout und kein Datenbankproblem auf. Der Flaschenhals ist unverändert die Backend-Parallelität, nicht die Datenbank. 750 gleichzeitige Nutzer sind unkritisch. Alle Angaben in Teil 1 sind Messwerte; Teil 2 enthält Hochrechnungen und Empfehlungen und stellt keine zugesicherte Kapazität dar.